K.R. Mangalam University, Gurugram

Data Structures (ETCCDS202)

Unit–1 Assignment: Foundations & Algorithmic Analysis

Course Details

Name of the School:	School of Engineering & Technology
Program:	B.Tech (AI and ML)
Course Title:	Data Structures
Course Code:	ETCCDS202
Unit Title:	Foundations & Algorithmic Analysis
Student Name:	Taruna Tewatia
Roll Number:	2501730115
Section:	A
Semester:	2
Batch:	2025-26
Submitted To:	Mrs. Neetu Chauhan
Submission Date:	5 March 2026

Introduction

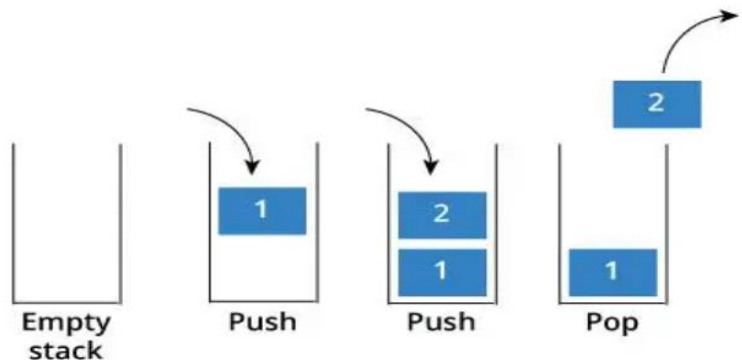
In this assignment, I implemented a small toolkit in Python to understand the concepts of recursion, algorithmic analysis, data structures, and algorithm paradigms. The project includes implementation of Stack ADT, recursive factorial, Fibonacci (naive and memoized), Tower of Hanoi, and recursive binary search. The time and space complexities of each algorithm are also analyzed.

Stack ADT

What is an ADT?

An Abstract Data Type (ADT) defines the behavior of a data structure by specifying the operations that can be performed on it, without describing how these operations are implemented internally.

For example, a Stack ADT allows operations like **push, pop, peek, is_empty, is_full, size**



Difference Between Data Structure and ADT

An **Abstract Data Type (ADT)** defines what operations can be performed on data. It describes the behavior of the structure but does not explain how it is implemented internally.

A **Data Structure**, on the other hand, is the actual implementation of the ADT in memory. It shows how data is organized and how the operations are executed.

For example, a Stack ADT defines operations like push, pop, peek, and is_empty. However, the stack can be implemented using an array or a linked list. The ADT describes what the stack does, while the data structure describes how it is built.

Why are Stack Operations O(1)?

Stack operations like push and pop are usually constant time O(1) because they operate only on the top element of the stack. No traversal of elements is required, so the time taken does not depend on the size of the stack.

Recursive Factorial

Algorithm Explanation

Factorial of a number n is defined as:
$n! = n \times (n - 1)!$

Base Case:
fac(0) = 1 or fac(1) = 1

Recursive Case:
fac(n) = n × factorial(n − 1)

Time & Space Complexity

Each recursive call reduces n by 1.

Time Complexity: O(n)
Space Complexity: O(n)

Best Case: O(1) when n = 0
Worst Case: O(n)

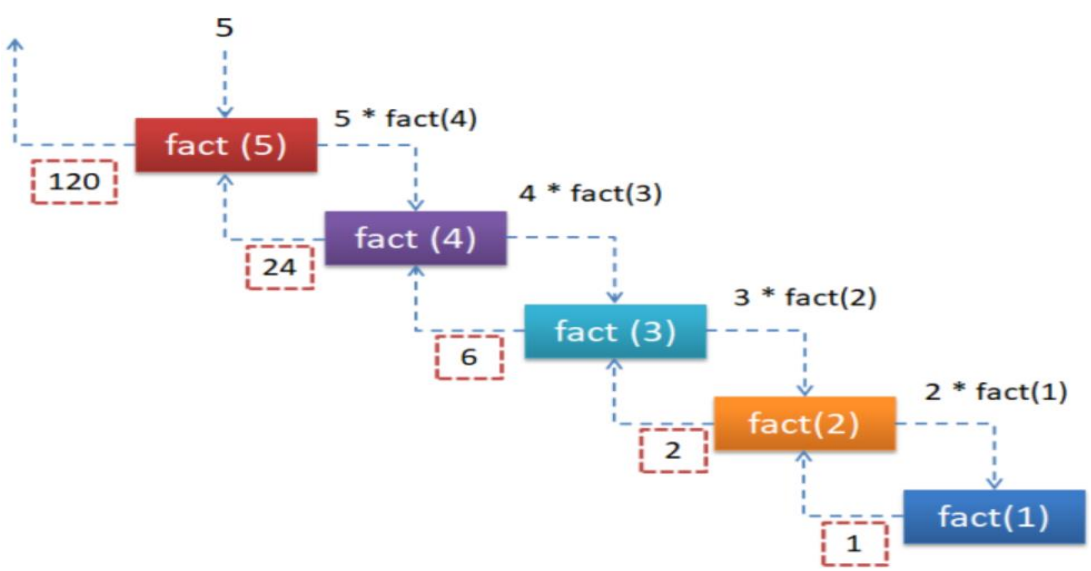
Understanding the Call Stack in Recursion

In recursion, every time a function calls itself, the system stores that function call in a special memory area called the **call stack**.
Each recursive call:

- Gets its own copy of local variables
- Is pushed onto the stack
- Waits until the deeper recursive call finishes

When the base case is reached, the function stops calling itself and starts returning values. The stored function calls are then removed one by one from the stack in reverse order (**Last In First Out – LIFO**).

For example: factorial(5)



Fibonacci (Naive and Memoized)

Naive Recursive Fibonacci

Explanation

The naive Fibonacci implementation recursively computes:
$F(n) = F(n-1) + F(n-2)$
This leads to repeated calculations of the same subproblems.

Time & Space Complexity

Time Complexity: O(2^n)
Space Complexity: O(n)

Why Naive Fibonacci becomes slow for large n?

Naive Fibonacci becomes slow for large n because it repeatedly recomputes the same subproblems due to recursive calls.
Its time complexity is exponential ($O(2^n)$), which grows very rapidly as n increases.

Memoized Fibonacci (Dynamic Programming)

Explanation

In memoized Fibonacci, previously computed results are stored in a dictionary (memo). If the value is already computed, it is returned directly instead of recalculating.
This avoids repeated work.

Time & Space Complexity

Time Complexity: O(n)
Space Complexity: O(n)

Why is Memoization related to Dynamic Programming?

Memoization stores solutions to smaller subproblems and reuses them when needed. This is the main idea behind Dynamic Programming — solving overlapping subproblems efficiently.

Tower of Hanoi

Explanation

Tower of Hanoi is a classic recursive problem where:

1. Move n−1 disks to auxiliary rod
2. Move nth disk to destination
3. Move n−1 disks to destination

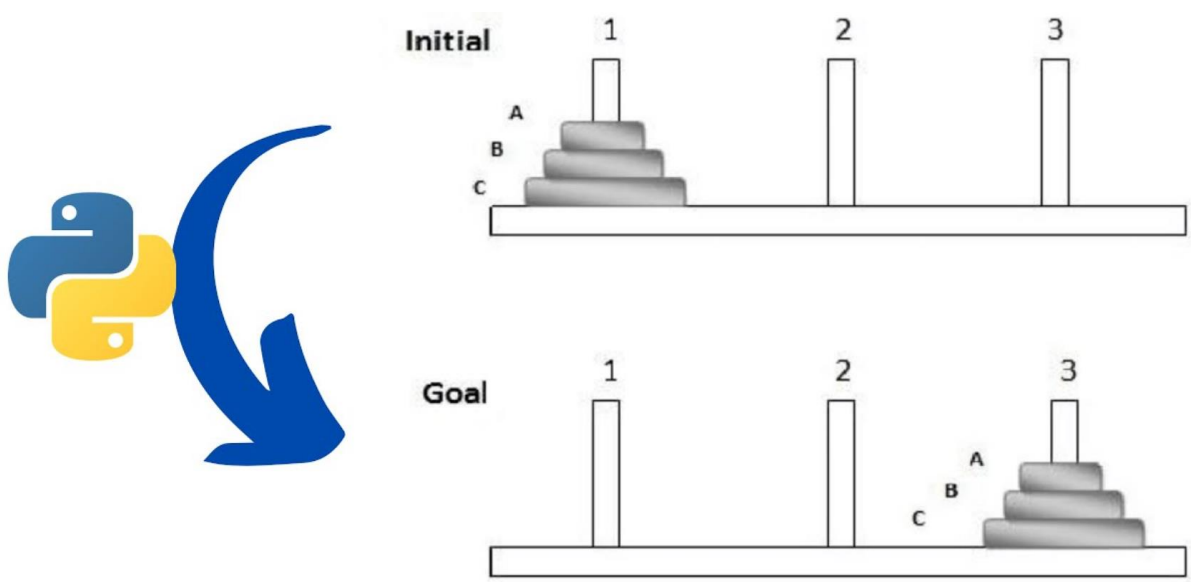
Base Case:
If n = 1, directly move the disk.

Why number of moves become $2^n - 1$?

The number of moves becomes $2^n - 1$ because to move n disks, we must move n−1 disks twice plus one move for the largest disk, which forms the recurrence $T(n) = 2T(n-1) + 1$

Complexity

Time Complexity: O(2^n)
Space Complexity: O(n)



Recursive Binary Search

Time Complexity of Recursive Binary Search

Binary search works by dividing the array into two halves at every step.
At each recursive call:

- We check the middle element → takes **O(1)** time
- Then we search only one half of the array → size becomes **n/2**

So the recurrence relation becomes:
$T(n) = T(n/2) + O(1)$

Solving the Recurrence (Conceptually)

Each step reduces the problem size to half:
$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \ldots \rightarrow 1$
How many times can we divide n by 2 until it becomes 1?
Answer: **$\log_2 n$ times**
So total time complexity becomes:
$T(n) = O(\log n)$

Best, Average, and Worst Cases

Best Case – O(1)

When the key is exactly the middle element in the first comparison.
Only one comparison is needed.

Average Case – O(log n)

Usually, the element is found after dividing the array several times.
On average, we divide the array about log n times.

Worst Case – O(log n)

When:

- The element is not present
  OR
- It is located at the extreme end of the array

In this case, we keep dividing until the array size becomes 1.
So maximum comparisons ≈ log n.

Space Complexity

Since this is recursive, function calls are stored in the call stack.
Maximum recursion depth = log n. So, space complexity = O(log n)

Short Paradigm Reflection

Binary Search as Divide & Conquer

Binary Search follows the Divide & Conquer paradigm because it repeatedly divides the problem into smaller subproblems. Instead of searching the entire array, it checks the middle element and eliminates half of the array in each step. After dividing, it recursively searches only in the relevant half. This process continues until the element is found or the search space becomes empty. Since the problem size reduces by half at every step, it clearly follows the divide, solve, and combine pattern of Divide & Conquer.

Fibonacci (Memoized) as Dynamic Programming

The normal recursive Fibonacci function recomputes the same values multiple times. For example, while calculating fib(5), the function calculates fib(3) and fib(2) repeatedly. This causes unnecessary repeated work and increases time complexity exponentially.

Memoization solves this problem by storing already computed Fibonacci values in a dictionary (or memory). When the function needs a value again, it directly returns the stored result instead of recomputing it. This eliminates overlapping subproblems and reduces time complexity from $O(2^n)$ to $O(n)$.

Because it stores solutions of subproblems and reuses them, memoized Fibonacci follows the Dynamic Programming paradigm.

Tower of Hanoi and Recursion

Tower of Hanoi is a classic example of recursion. The main idea is to solve a smaller version of the same problem.

To move n disks:

1. Move $n-1$ disks to the auxiliary rod.
2. Move the largest disk to the destination rod.
3. Move the $n-1$ disks from auxiliary to destination.

The base case occurs when there is only one disk. In that case, it can be directly moved to the destination rod in one step.

So, the problem keeps reducing to a smaller subproblem (n−1 disks) until it reaches the base case. This repeated self-calling structure makes Tower of Hanoi a pure recursive problem.

Real-Life Examples of Paradigms

Divide & Conquer (Real-Life Example)

Searching for a word in a dictionary.
Instead of reading page by page, we open near the middle, then eliminate half the dictionary based on alphabetical order. This is similar to binary search.

Dynamic Programming (Real-Life Example)

Planning travel routes while saving fuel costs.
If we already know the shortest path between certain cities, we store that information and reuse it instead of calculating it again. This saves time and effort, similar to memoization.

Greedy (Real-Life Example)

Giving change in currency.
When returning money, we usually give the largest denomination first (like ₹500 before ₹100). This makes a locally optimal choice at each step. That is the greedy approach.